Phase 2 IPOPT Callback Timing Profile

Empirical Cost Breakdown of the Perimeter-Refinement Solver

May 2026

Abstract

This document records empirical timing measurements collected by the `ProfilingState` instrumentation in `src/profiling.py`. Each row of Table 2 reports the wall-clock cost and the Steiner-recomputation count of one IPOPT callback (objective, gradient, constraints, Jacobian, Hessian) for a single Phase 2 run on a 10-cell torus. The data identifies the Hessian callback — specifically the finite-difference Steiner contribution documented in §6 of `01-phase2-derivatives` — as the sole performance bottleneck, consuming 98.41% of total wall time. This document is a *living artefact*: Section 6 will be extended as profiles for other mesh resolutions and cell counts are collected.

Contents

1 Overview

Phase 2 of the `surface-partition` pipeline solves the constrained nonlinear program (1) using IPOPT [1]. At each interior-point iteration IPOPT invokes five callbacks:

Callback	Quantity returned
`objective`	perimeter value $P(\boldsymbol{\lambda})$
`gradient`	perimeter gradient $\nabla P(\boldsymbol{\lambda})$
`constraints`	area vector $(A_0, \ldots, A_{N-2})(\boldsymbol{\lambda})$
`jacobian`	area Jacobian $J(\boldsymbol{\lambda})$
`hessian`	Lagrangian Hessian $H = \sigma\,\nabla^2 P + \sum_k \mu_k\,\nabla^2 A_k$

The derivations of all five quantities are given in document `01-phase2-derivatives`; the present document reports only their *measured cost*. File I/O (HDF5 checkpoint reads and writes), topology migrations, and HDF5 checkpointing are excluded from all figures reported here.

$$\min_{\boldsymbol{\lambda}\in\mathbb{R}^n} \; P(\boldsymbol{\lambda}) \quad \text{subject to} \quad A_k(\boldsymbol{\lambda}) = \bar{A}, \quad k = 0, \ldots, N-2 \tag{1}$$

2 Profiling Methodology

2.1 What is recorded

The class `ProfilingState` (`src/profiling.py`) accumulates two quantities per callback type across all IPOPT iterations and all topology cycles within a single run:

- **Wall-clock time** — total and per-call mean for each callback.
- **Steiner recomputation count** — a running integer incremented each time `record_steiner()` is called inside a callback. One increment corresponds to one full evaluation of the Steiner geometry (Fermat-Torricelli point and associated perimeter or area) for all active triple points. This counter directly measures the number of finite-difference perturbations executed within a callback, making it the primary diagnostic for the FD cost.

Results are written to `timing_profile.yaml` in the campaign directory when `--profile` is passed to `scripts/refine_perimeter.py`.

Code: profiling accumulator

`src/profiling.py`: `ProfilingState`
`record(key, dt)` — accumulate wall time.
`record_steiner(key, n)` — increment the Steiner recomputation count.
`finalize()` — compute means and percentage breakdown.
`to_yaml_dict()` — serialize to `timing_profile.yaml`.

2.2 Why only Jacobian and Hessian report Steiner evaluation counts

All Steiner derivative quantities — the perimeter gradient, the area Jacobian, and both Hessians — are computed by finite differences (see document `01-phase2-derivatives`, §6). None of these quantities has an analytical derivative implementation. However, the Steiner recomputation count is only recorded for the Jacobian and Hessian callbacks. This is an instrumentation choice: the profiling handle `_prof` is threaded into the vectorized Steiner functions in `src/partition/vectorized_steiner.py` only when they are called from those two paths. In the gradient callback, the same finite-difference loop executes and the Steiner geometry is re-evaluated

for every Steiner-affected VP, but `_prof` is not passed in, so those evaluations are not counted. The gradient's Steiner FD cost is real but small (absorbed into the 0.54% wall time in Table 2); the counter was wired primarily where the cost was expected to be large.

Remark 2.1 (FD structure and Steiner evaluation counts)**.** A single gradient call perturbs each of the $\sim$180 Steiner-affected VPs once (forward FD, $\varepsilon = 10^{-6}$) and evaluates the Steiner geometry at each perturbation — but these are not counted. A single Jacobian call does the same for the area constraints, and *is* counted: $\sim$61 evaluations for a run with 20 triple points. A single Hessian call then re-runs the full FD gradient and Jacobian once per Steiner-affected VP (the outer FD layer), giving $180 \times 61 \approx 11\,000$ counted evaluations per Hessian call — in agreement with the measured 11 041.

3 Reference Run

All numbers in Section 4 come from a single baseline run whose parameters are given in Table 1. This run used the exact Hessian option (`exact_hessian: true`) and `allow_partial_convergence: true`, which allows the pipeline to proceed to topology migration even when IPOPT exits with a non-fatal status code.

Table 1: Reference run parameters.

Parameter	**Value**
Surface	torus ($R = 1$, $r = 0.4$)
Number of cells N	10
Mesh vertices	242 608
Active VPs	7 110
Triple points	20
Solver	IPOPT, exact Hessian
IPOPT iterations	369 (across 2 topology cycles)
Wall-clock total	1 284.32 s
Campaign directory	`ipopt_btol0.001_lbfgs30_hess_bestiter_partial`

4 Results

Table 2 gives the full per-callback breakdown extracted from `timing_profile.yaml` for the reference run of Section 3.

Table 2: Per-callback timing breakdown, reference run ($N = 10$, 242 608 vertices).

Callback	**Calls**	**Total (s)**	**% wall**	**Mean (s/call)**	**Steiner evals/call**
Objective	1 724	1.00	0.08	0.0006	—
Gradient	372	6.95	0.54	0.019	—
Constraints	1 724	2.46	0.19	0.0014	—
Jacobian	372	8.26	0.64	0.022	61
Hessian	368	1 263.95	**98.41**	3.43	11 041
Total	—	1 284.32	100.00	—	4 085 780

5 Analysis: the Hessian Bottleneck

The Hessian callback dominates runtime by a factor of ~ 150 over the next most expensive callback (Jacobian at 0.64%). The cause is the nested finite-difference scheme used for the Steiner contributions $\nabla^2 P_{\text{st}}$ and $\nabla^2 A_k^{\text{st}}$, described in §6.4 and §6.5 of document `01-phase2-derivatives`.

Cost decomposition. Let T denote the number of active triple points and $n = |S|$ the total number of Steiner-affected VPs (the set `tp_affected_vps` in `src/partition/partition_arrays.py`). Each triple-point triangle has exactly 3 boundary edges, each carrying one VP, so $n = 3T$. In the reference run $T = 20$, giving $n = 60$.

A single Jacobian call evaluates the Steiner geometry once as a base, then once per VP in S, for a total of $n + 1$ recomputations — in agreement with the measured 61.

The Hessian executes two nested FD loops. The perimeter Hessian (§6.4 of `01-phase2-derivatives`) uses central differences: for each of the n VPs in S it calls the gradient FD twice ($+\varepsilon$ and $-\varepsilon$), and each gradient call costs $n + 1$ recomputations. The area Hessian (§6.5) uses forward differences: one base Jacobian call plus n perturbed Jacobian calls, each costing $n + 1$ recomputations. The total is

$$\text{Steiner evaluations per Hessian call} = \underbrace{2n(n+1)}_{\nabla^2 P_{\text{st}}} + \underbrace{(n+1)^2}_{\nabla^2 A_k^{\text{st}}} = (n+1)(3n+1) \tag{2}$$

With $n = 60$: $(61)(181) = 11\,041$ — in exact agreement with the measured value.

Remark 5.1 (Quadratic scaling with triple-point count)**.** Since $n = 3T$, Equation (2) gives $(n+1)(3n+1) \approx 3n^2 = 27T^2$ for large T. The Hessian cost therefore scales *quadratically* with the number of triple points. Increasing the number of cells N increases T and therefore the cost grows as $\mathcal{O}(T^2)$. This is the key motivation for implementing an analytical Steiner Hessian.

Why the exact Hessian is required. An L-BFGS quasi-Newton approximation could avoid the FD Hessian cost entirely, but in practice the L-BFGS approximation does not converge tightly enough to push VPs to the mesh boundary, so topology migrations are never triggered. The exact Hessian is therefore required for production runs that need to discover the correct partition topology.

6 Scaling Outlook

This section will be extended as additional runs are profiled. Two scaling axes are planned:

- **Mesh refinement.** Fix $N = 10$, vary the torus mesh resolution (n_θ, n_ϕ), and record Table 2-style breakdowns.
- **Cell-count scaling.** Fix a moderate mesh resolution, vary N, and measure how the number of triple points and the Hessian Steiner evaluation count scale with N.

Summary table

Callback	**Current cost (reference run)**	**Planned optimization**
Objective, constraints	$< 0.3\%$ of wall; negligible	none
Gradient, Jacobian	$\sim 1\%$ of wall; fast	none
Hessian	98.41% of wall; FD Steiner dominates	Analytical Steiner Hessian (see `01-phase2-derivatives` §6 for the FD baseline)

References

[1] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.